Big Data Analytics of Crime Patterns using Hadoop MapReduce and Apache Hive

Project Review 1

Your Group Members

Amrita Vishwa Vidyapeetham

August 12, 2026

Outline

Problem Statement & Motivation

Problem Statement:

- Analyzing massive crime datasets using traditional sequential systems is inefficient.
- Need a scalable pipeline to calculate crime frequencies and perform advanced queries.

Motivation:

- Enables data-driven decision making for public safety.
- Optimizes police resource allocation through pattern recognition.

Dataset Introduction

Chicago Crimes (2001 to Present)

- **Source:** Chicago Data Portal API
- **Size:** 10,000 recent records extracted for processing (meets 5,000+ requirement)
- **Key Attributes:**
  - `id`, `case_number`, `date`
  - `primary_type`, `description`
  - `arrest`, `domestic` (Boolean flags)
  - `district`, `ward`, `location_description`
- **Preprocessing:** Cleaned embedded newlines from CSV to ensure correct MapReduce parsing.

System Architecture & HDFS Storage

Pipeline Overview:

1. **Ingestion:** Dataset uploaded to HDFS (`/user/$USER/bigdata_project/crimes`)
2. **MapReduce:** Parallel processing of crime records to aggregate frequencies.
3. **Hive:** Data loaded into Hive via `OpenCSVSerde` for complex SQL analytics.

(Insert HDFS Screenshot Here)

MapReduce: Crime Type Counting

Goal: Calculate frequency of each primary crime type.

Mapper Logic:

- Parses each CSV line, handling quotes safely.
- Emits `(PrimaryType, 1)`.

Reducer Logic:

- Iterates over values for each key.
- Emits `(PrimaryType, TotalCount)`.

MapReduce Execution

(Insert MapReduce Execution & Output Screenshots Here)

- **Input:** Cleaned CSV on HDFS.
- **Output:** Text file containing counts (e.g., `THEFT: 2150`).

Hive Table Creation

- Loaded dataset into Hive table `crimes`.
- Used `OpenCSVSerde` to accurately parse quoted CSV fields.
- Created a secondary `community_areas` reference table for joins.

Hive Queries Demonstrated

Successfully implemented all required SQL clauses:

- **SELECT & WHERE:** Filtering narcotic arrests.
- **ORDER BY:** Chronological listing of homicides.
- **GROUP BY & COUNT:** Crimes per location description.
- **HAVING:** Filtering crime types $>$ 500 occurrences.
- **SUM & AVG:** Aggregating total arrests and average domestic crimes.
- **MAX & MIN:** Finding safest and most dangerous districts.
- **JOIN:** Displaying actual community names instead of just codes.

Hive Execution Results

(Insert Hive Query Screenshots Here)

Conclusion

- Successfully demonstrated a complete Big Data pipeline.
- **MapReduce** provided efficient, low-level data aggregation.
- **Apache Hive** provided a powerful, SQL-like abstraction for business intelligence on massive datasets.
- This project highlights how Big Data technologies scale to solve real-world law enforcement analytics problems.

Thank You!

Questions?